

1.3 Technology Stack Evaluation

Project: E-Learning Web Pages for Computer Science
Student: Zain Chohan (32001784)

Objective

The technology stack must enable the creation of **interactive, browser-based visualisations** of CPU pipeline operation while remaining lightweight, accessible, and easily maintainable. The system must simulate a simplified 5-stage instruction pipeline (IF, ID, EX, MEM, WB) with hazards, forwarding, and performance metrics. It should run entirely in a web browser without requiring server installation, ensuring accessibility for all students.

Platform Decision

Chosen Platform: Web (HTML5 / CSS3 / JavaScript)

Rationale:

- Universally accessible across devices (no installation required).
- Enables immediate interaction for students and instructors.
- Compatible with open educational hosting platforms such as GitHub Pages, Netlify, or University servers.
- Facilitates integration with the Student Learning Experience Assessment (SLEA) questionnaire.

Alternatives Considered:

Desktop-based platforms (Java/Qt) and mobile applications (Android/iOS) were rejected because they reduce accessibility, require additional maintenance, and complicate distribution across systems.

Core Technologies

Component	Technology	Purpose / Justification
-----------	------------	-------------------------

Frontend Base	HTML5 / CSS3 / JavaScript (Vanilla)	Provides structure, styling, and interactivity with maximum browser compatibility.
UI Framework	React.js (<i>optional</i>)	Enables component-based architecture for managing interactive UI updates.
Graphics / Visualisation	Canvas API or SVG with D3.js	Used to render the pipeline stages and animate instruction flow. Canvas provides high performance; D3.js is suited for data-driven updates.
Code Editor Integration	Monaco Editor	Offers syntax highlighting for MIPS-like assembly input.
Testing	Jest	Provides lightweight unit testing for pipeline logic validation.
Deployment / Hosting	GitHub Pages / Netlify / University Web Server	Enables free, public, and academic hosting for static web applications.

Supporting Libraries & Tools

- **Chart.js / Recharts:** Visualise CPI, stalls, and performance metrics.
 - **Tailwind CSS / Bootstrap:** Ensure responsive and professional user interface design.
 - **Framer Motion:** Enable smooth animations for instruction movement.
 - **ARIA / WCAG Utilities:** Maintain accessibility compliance for all users.
-

Programming Language Choice

Language: JavaScript (ES6+)

- Browser-native and widely supported — no external runtime required.
- Provides asynchronous control for simulation step and timing logic.
- Enables fast DOM updates for pipeline visualisation and state changes.

Alternative Considered: TypeScript — may be introduced later to add static typing for larger-scale maintainability.

Architecture Overview

The system follows an **MVC-style (Model–View–Controller)** design pattern:

- **Model:** Simulated CPU state including instruction queue, registers, memory, and hazards.
- **View:** Graphical representation of pipeline stages using Canvas or SVG.
- **Controller:** User interaction management — start, step, pause, and speed control, plus hazard visualisation.

This separation allows modular development, easy debugging, and future scalability.

Development Environment

- **IDE:** Visual Studio Code
 - **Version Control:** GitLab repository (University of Reading)
 - **Browser Testing:** Chrome, Firefox, Edge
 - **Documentation & Collaboration:** Google Docs (e-logbook, literature review, feedback tracking)
-

Risk & Feasibility Analysis

Risk	Mitigation Strategy
Canvas performance bottlenecks	Optimise draw loops using <code>requestAnimationFrame</code> ; reduce unnecessary re-rendering.
Complex hazard logic	Develop and test using isolated JS modules before full integration.

Accessibility compliance	Validate regularly using WCAG tools and screen reader testing.
Browser compatibility	Test across Chrome, Firefox, and Edge early in development.
Time management	Follow Gantt chart milestones; use iterative sprints.

Summary

The chosen technology stack provides a robust foundation for creating an **interactive, web-based CPU pipeline simulator**. Using **JavaScript, Canvas/SVG**, and modern open-source frameworks ensures performance, accessibility, and maintainability. The system supports both **interactive learning** and the **SLEA questionnaire** component for evaluating user experience and educational effectiveness.